

Requisiti

Versione	Data	Descrizione
1.0	25 Settembre 2026	Prima versione del documento Requisiti

Qualità del software

Per garantire la qualità del software viene adottata la tassonomia di **McCall**, focalizzata sulle seguenti dimensioni prioritarie:

- **Funzionamento del prodotto (*Product Operation*):**
 - **Correttezza:** copertura totale dei requisiti tramite *User Story* definite nel *Product Backlog*, ciascuna corredata da criteri di accettazione ben definiti e verificabili
 - **Affidabilità:** continuità operativa dell'applicazione garantita da una gestione rigorosa delle eccezioni di rete e di gioco, affiancata da un sistema di logging degli eventi critici
 - **Usabilità:** interfaccia grafica intuitiva, in grado di restituire un feedback visivo immediato sullo stato delle risorse (HP ed energia) e di evidenziare chiaramente le azioni valide per il turno corrente
- **Revisione del prodotto (*Product Revision*):**
 - **Manutenibilità:** codice sorgente ordinato, modulare e documentato, aderente alle convenzioni del linguaggio *Java* per facilitare interventi correttivi ed evolutivi
 - **Testabilità:** verificabilità della logica di dominio (calcolo dei danni, consumo di energia, ordine di risoluzione degli effetti e turnazione) tramite suite di test unitari con *JUnit*
- **Transizione del prodotto (*Product Transition*):**
 - **Portabilità:** esecuzione multiplatforma del *client* (e del *server*) garantita dall'astrazione hardware fornita dalla *Java Virtual Machine (JVM)*

Ingegneria dei requisiti

Elicitazione dei requisiti

L'elicitazione dei requisiti è stata condotta attraverso un processo strutturato in:

- **Analisi del dominio di gioco:** studio formale delle meccaniche tipiche dei *Collectible Card Games (CCG)* per estrarre le regole di business (gestione dei turni, risorsa energetica, calcolo dei danni, risorsa vitale e risoluzione degli effetti)

- **Modellazione degli scenari d'uso:** formalizzazione dei comportamenti attesi del sistema in linguaggio naturale attraverso *User Story* e descrizioni analitiche dei flussi di gioco
- **Brainstorming e definizione dell'architettura:** sessioni di analisi tecnica per la definizione dei vincoli di rete (*Client/Server*), della persistenza dei dati e delle interazioni dell'interfaccia grafica

Ovviamente il nostro team simulerà i bisogni e le richieste dell'azienda proprietaria e impersonificherà il *Product Owner*.

Specifica dei requisiti

Per la specifica dei requisiti utilizzeremo lo standard **IEEE 830** che ne definisce la struttura:

1. Introduzione
 - 1.1. Obiettivo: la presente specifica dei requisiti ha lo scopo di descrivere in modo formale, strutturato e univocamente verificabile i requisiti funzionali, non funzionali e i vincoli di sistema dell'applicazione *Thearchis*
 - 1.2. Scopo: consentire a utenti remoti di disputare partite multiplayer online al gioco *Thearchis*, gestendo l'autenticazione, la composizione dei mazzi e la sincronizzazione dello stato di gioco in tempo reale
 - 1.3. Definizioni, acronimi e abbreviazioni:
 - JVM: *Java Virtual Machine*, ambiente di esecuzione che fornisce l'astrazione hardware per l'esecuzione di *bytecode Java* su piattaforme eterogenee
 - DB: *Database*, sistema di gestione della persistenza dei dati
 - IDE: *Integrated Development Environment (Eclipse IDE)*
 - GUI: *Graphical User Interface*, interfaccia grafica tramite cui l'utente interagisce con il sistema
 - CCG: *Collectible Card Game*, gioco di carte collezionabili basato sulla costruzione strategica di un mazzo (*deck-building*)
 - HP: *Health Points*, punti vita che determinano le condizioni di vittoria/sconfitta dei giocatori
 - ATK: *Attack*, attacco della carta
 - DEF: *Defense*, difesa della carta
 - TCP/Socket: Protocollo di comunicazione di rete orientato alla connessione utilizzato per lo scambio di dati tra Client e Server
 - 1.4. Riferimenti: regolamento del gioco *Thearchis*
2. Descrizione generale
 - 2.1. Prospettiva del prodotto: applicazione distribuita con architettura *Client/Server*, articolata nei seguenti tre moduli *Maven*:
 - *game-client*: gestione della GUI, rendering del campo di battaglia e gestione dell'input utente
 - *game-common*: modulo di dominio contenente la logica delle carte, le formule di calcolo danni, le strutture dati dei mazzi e i protocolli di messaggistica
 - *game-server*: gestione delle connessioni socket simultanee, arbitraggio dello stato delle partite (*matchmaking*), validazione delle giocate e persistenza su DB

2.2. Funzioni del prodotto

- Gestione degli account utente (autenticazione, registrazione)
- Gestione della collezione personale e della composizione dei mazzi
- Gestione del matchmaking e creazione delle stanze di gioco per due giocatori
- Sincronizzazione del flusso di gioco in tempo reale (gestione fasi, risorsa energia, risoluzione attacchi ed effetti carte)

2.3. Caratteristiche dell'utente

- Admin: amministratore del sistema, responsabile del monitoraggio del server e della gestione delle utenze.
- Player: utente finale che accede all'applicazione per gestire i mazzi e giocare online

2.4. Vincoli

- Il sistema utilizza un'architettura *client/server* e comunica tramite *Java Socket (TCP)*
- Il codice è scritto in linguaggio *Java*

2.5. Presupposti e dipendenze

- È strettamente necessaria una connessione Internet stabile e attiva sia per i *Client* che per il *Server* per garantire la sincronizzazione in tempo reale dello stato delle partite
- *Server* in Cloud: il sistema dipende dalla disponibilità e raggiungibilità continuativa dell'applicazione Server ospitata su un provider Cloud
- *Database* in Cloud: la persistenza dei dati (account utente, credenziali, collezione di carte possedute e mazzi salvati) è affidata a un *database* gestito in Cloud; l'applicazione dipende dalla sua raggiungibilità remota via driver
- L'applicazione client/server dipende dal corretto scaricamento e risoluzione delle librerie esterne definite nel file *pom.xml* tramite *Apache Maven*

3. Requisiti specifici

3.1. Requisiti dell'interfaccia esterna

3.1.1. Interfaccia utente

- Login e registrazione: form di autenticazione e creazione nuovo account con gestione messaggi di errore
- Menu principale e gestione mazzi: accesso alle modalità di gioco, consultazione della collezione e schermata di composizione mazzi
- Tavolo da gioco: rappresentazione del campo di battaglia, carte in mano e sul terreno, indicatori di *HP* ed energia residua di entrambi i giocatori
- Pulsanti di azione: comandi contestuali abilitati durante il turno

3.1.2. Interfaccia hardware

- Client: PC con supporto a un ambiente di esecuzione *JVM* e dotazione di periferiche di controllo (mouse e tastiera)
- Server: *server* cloud con risorse di calcolo e memoria adeguate ad eseguire il processo server e gestire le connessioni socket concorrenti

3.1.3. Interfaccia software

- Librerie di rete: *API Java standard (java.net)* per la gestione delle comunicazioni *Client/Server* su protocollo *TCP*
- Database cloud (*DBMS*): Sistema di gestione dati remoto per la persistenza dei dati utente
- Driver JDBC: per consentire la connessione e l'esecuzione di query dal *Server* verso il *Database*

3.2. Richieste funzionali

3.2.1. Classe Client

ID	Requisito	Descrizione
1	Creazione account	L'utente deve poter creare un nuovo account oppure accedere ad uno già esistente (nickname univoco)
2	Collezione	L'utente deve poter inviare al server richieste sulla propria collezione
.1	Polvere	L'utente deve poter usare la polvere per creare carte. Questa si ottiene distruggendo le carte
3	Mazzi	L'utente deve poter accedere al deck builder: potrà costruire fino a 4 mazzi, uno per ogni eroe disponibile.
.1	Mazzo da battaglia	L'utente deve poter selezionare il mazzo che desidera usare in battaglia
4	Negozi	L'utente deve poter inviare al server richieste sul negozio
.1	Monete	L'utente deve poter usare le monete per comprare pacchetti di carte. Queste si ottengono vincendo le partite (10 monete a vittoria)
.2	Pacchetti	Ogni pacchetto ha 5 carte casuali, scelte in base alla probabilità di comparsa delle carte del gioco
5	Cerca partita	L'utente deve poter cercare una partita entrando in una lobby, in modo da sfidare un altro giocatore che sta cercando una partita
6	Flusso di gioco	Deve poter gestire il flusso di gioco rimanendo sincronizzato con il server
7	Resa	L'utente deve poter concedere la partita all'avversario in qualsiasi momento (arrendendosi), tornando al menù principale
8	Profilazione	L'utente deve poter accedere ai dati del proprio account (vittorie, credito, ecc)

3.2.2. Classe Player

ID	Requisito	Descrizione
1	Azioni di gioco	Le azioni di gioco saranno implementate sulla GUI come bottoni e menu a tendina: rappresentano le scelte che il giocatore può effettuare durante il suo turno, in diverse condizioni.
.1	Giocare carte	L'utente deve poter giocare le carte in qualsiasi momento, nel suo turno, se il suo mana rimanente glielo permette.

		Il costo in mana della carta da utilizzare non deve superare il mana rimanente del giocatore che la usa: una volta usata il suo costo in mana viene sottratto al mana rimanente del giocatore che la usa. - Per le creature l'azione è di evocazione e avviene cliccando sulla carta in mano (mouse sx) e trascinandola su uno slot libero del proprio lato del campo tenendo premuto il tasto sx del mouse - Per le magie l'azione è di attivazione e avviene cliccando sulla carta in mano (mouse sx) e scegliendo dal menù a tendina l'opzione "Attiva". Se l'effetto lo richiede dovrà scegliere la/le creatura/e bersaglio dal terreno, altrimenti l'effetto si risolverà in automatico
.2	Potere divino	L'utente deve poter usare il potere del suo eroe una volta per turno in qualsiasi momento, nel suo turno, se il suo mana rimanente glielo permette. Una volta usato il potere, il suo costo viene sottratto al mana rimanente del giocatore che lo usa
.3	Attaccare	L'utente deve poter attaccare con qualsiasi creatura in suo possesso (dal suo lato del campo) in qualsiasi momento, nel suo turno, se la carta ha $ATK > 0$. L'attacco può avere come bersaglio solo le creature avversarie, e quando il campo avversario è vuoto allora anche il giocatore avversario, riducendo i suoi <i>HP</i> dello stesso valore dell'attacco della creatura attaccante Durante l'attacco le creature che partecipano al combattimento subiscono un decremento della propria difesa pari al valore dell'attacco della creatura avversaria: se la difesa risulta ≤ 0 allora la creatura viene distrutta
.4	Fine turno	L'utente deve poter passare il turno in qualsiasi momento, nel suo turno
.5	Caratteristiche carta	L'utente deve poter vedere le caratteristiche della carta in modo completo (<i>Effetto, Costo, ATK, DEF, Tipo</i>) cliccando con il tasto dx del mouse sulla carta in mano o sul terreno
2	HP	Ogni giocatore inizia con 30 punti vita, che dovranno essere sempre visibili ad entrambi i giocatori durante la partita. Quando un giocatore ha $HP \leq 0$ ha perso la partita
3	Mano	Ogni giocatore avrà la sua mano iniziale composta da 5 carte, pescata casualmente dal proprio mazzo. La mano può avere un massimo di 10 carte, quelle che vengono pescate in eccesso verranno scartate
4	Pescata	La pescata è automatica all'inizio del proprio turno e permette di pescare la carta in cima al mazzo. Il primo turno della partita non pesca (quindi il giocatore che gioca per secondo pesca, il primo no)
5	Mana	L'utente inizia la partita con 1 di mana massimo, che aumenta di 1 ogni suo inizio turno fino ad un limite massimo di 10

3.2.3. Classe Server

ID	Requisito	Descrizione
1	Turnazione	La turnazione è definita casualmente
2	Mazzo di carte	Deve poter gestire il mazzo di carte

.1	Numero di carte	Il mazzo sarà composto da 30 carte, ciascuna carta deve essere presente al massimo in doppia copia
.2	Fatica	Quando il mazzo finisce, la prima pescata con il mazzo vuoto toglie 1 HP al giocatore che pesca. Ad ogni pescata successiva il danno aumenta di 1
.3	Shuffle	Deve prevedere una funzione per mescolare il mazzo di carte
3	Numero giocatori	Deve gestire 2 giocatori al massimo, non permettendo ad altri client di connettersi
4	Flusso di gioco	Deve poter gestire il flusso di gioco comunicando con i client
5	Negozi	Deve poter gestire e validare le richieste di negozio facendo da tramite tra DB e client
6	Collezione	Deve poter gestire e validare le richieste di collezione facendo da tramite tra DB e client
7	Mazzi	Deve poter gestire e validare le richieste di creazione mazzi facendo da tramite tra DB e client
8	Validazione	Deve poter gestire e validare le richieste in partita dei client
9	Timeout resa	Se un giocatore non gioca il suo turno concede in automatico la partita dopo 1 minuto
10	Disconnessione	Deve essere gestita con il <i>timeout resa</i> . Se entrambi i giocatori si disconnettono prima del timeout la partita si chiude senza vincitore.

3.2.4. Classe Database

ID	Requisito	Descrizione
1	Connessione	La classe deve permettere di effettuare la connessione al db remoto (cloud)
2	Metodi query	Deve fornire tutti i metodi per interrogare il db, gestendo tutte le eccezioni
3	Dati utente	Deve memorizzare i dati di ogni giocatore
.1	Username/Password	Memorizzare il nome utente e la password
.2	Collezione	Memorizzare la collezione
.3	Mazzi	Memorizzare i mazzi. Devono avere 30 carte esatte, ed ogni carta può essere usata al massimo in doppia copia. Può contenere solo carte <i>generiche</i> e dell'eroe scelto
.4	Monete	Memorizzare le <i>monete</i> per comprare pacchetti
.5	Polvere	Memorizzare la <i>polvere</i> per creare le carte
.6	Statistiche	Memorizzare le statistiche della storia del giocatore

4	Master set	Deve memorizzare la lista di tutte le carte del gioco
---	------------	---

3.2.5. Classe Eroe

ID	Requisito	Descrizione
1	ID	Deve avere un ID identificativo (ci sono 4 eroi ed una classe generica)
2	Nome	Deve avere un nome tra: - <i>da definire gli altri 4</i> - Generico
3	Potere	Deve avere un potere
4	Costo Potere	Il potere deve avere un costo in mana

3.2.6. Classe Carte

ID	Requisito	Descrizione
1	ID	Ogni carta deve avere un suo ID univoco (coppie di carte uguali hanno stesso ID)
2	Nome	Ogni carta deve avere un nome
3	Costo mana	Ogni carta deve avere un costo in mana compreso tra 0 e 10 (compresi)
4	Tipo	Una carta deve avere un tipo tra: - Creatura - Magia
.1	Sottotipo	Solo le creature hanno un sottotipo tra: - Bestia - Umano - Demone - Divino - Normale
.2	Summoning Sickness	Le creature non posso attaccare il turno nel quale vengono evocate, a meno che abbiano l'abilità " <i>corsa</i> "
5	ATK	Ogni creatura deve avere un valore di attacco
6	DEF	Ogni creatura deve avere un valore di difesa >0
7	Effetto	Una carta può avere un effetto
8	Abilità	Una carta con effetto può avere un'abilità (che viene considerata come effetto, quindi potrebbe non fare nient'altro) tra: - corsa: immune alla summoning sickness - altre da definire
9	Rarità	Una carta deve avere una rarità che descrive la probabilità di trovarla: - Comune (<i>% da definire</i>) - Rara (<i>% da definire</i>)

		- Epica (% <i>da definire</i>) - Leggendaria (% <i>da definire</i>)
10	Eroe	Ogni carta ha un <i>eroe</i> di appartenenza, oppure è generica. Le carte dell'eroe possono essere usate solo usando l'eroe corrispondente, quelle generiche possono essere usate in tutti i mazzi

3.2.7. Classe GUI [DA DEFINIRE]

<i>ID</i>	<i>Requisito</i>	<i>Descrizione</i>

3.3. Requisiti di prestazione

- Il sistema non deve presentare difetti di programmazione che portino al crash delle partite
- Sono tollerate perdite di connessione client/server

3.4. Attributi del sistema software

- Come descritti nel paragrafo (1)